

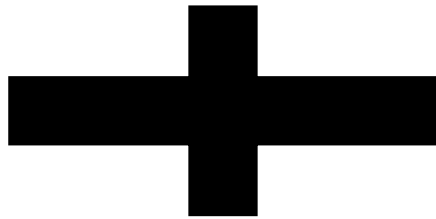


PF

FINAL PROJECT

REPORT:

NAME : EHTISHAM AHMED
ROLL NO: 2026(S)-CYS-95
SUBMITTED TO:MR.MUBASHIR
DATED : 24TH JUNE, 2026
PROGRAM : CYBER SECURITY
DEPARTMENT: COMPUTER
ENGINEERING.



MEDCORE

Medical Management System

A Desktop Application for Hospital & Clinic Management

Presented By:

EHTISHAM AHMED

Technologies Used:

Python 3 | PyQt5 | SQLite3

1. Introduction:

MedCore is a fully self-contained, single-file desktop application developed using Python 3 and the PyQt5 graphical user interface framework. It is designed to address the administrative and clinical management needs of small to medium-sized hospitals, clinics, and healthcare facilities.

The system stores all data locally using SQLite3, a lightweight and serverless relational database engine that ships as part of the Python standard library. This design choice makes MedCore easy to deploy and run on any machine without requiring a separate database server installation.

MedCore covers the full operational lifecycle of a healthcare facility — from patient registration and doctor management, through appointment scheduling, billing, and payment processing, to an intelligent symptom-analysis module that maps patient complaints to the most appropriate medical specialist.

2. Project Objectives:

The primary goals of the MedCore Medical Management System are:

- To provide a unified desktop platform for managing patients, doctors, appointments, and billing.
- To eliminate the need for paper-based records by digitalizing all clinical and administrative data.
- To enforce secure access control by requiring authenticated login with SHA-256 hashed passwords.
- To support multi-role access (Admin, Doctor, Staff) so that different users see only relevant features.
- To offer a rule-based Symptom Analyzer that recommends the right specialist based on a patient's complaints.
- To allow database backup at any time to prevent accidental data loss.
- To keep the deployment footprint minimal — a single Python file and one SQLite database file.

3. System Architecture:

MedCore follows a monolithic, single-tier desktop architecture. All application logic, the graphical user interface, and the data access layer reside within a single Python source file (main.py). The underlying database is a local SQLite file (medical_system.db) that persists on disk.

3.1 Architectural Layers

Layer	Component	Description
Presentation	PyQt5 Widgets	All windows, dialogs, tables, and input forms visible to the user.
Business Logic	Python Methods	Validation, symptom analysis, billing calculations, and workflow control.
Data Access	Database Class	All SQL queries, inserts, updates, and deletes via the sqlite3 module.
Storage	SQLite3 File	Persistent on-disk file storing all tables and records.

4. Database Design:

The system uses six relational tables stored in a single SQLite file. Each table is created automatically on first launch if it does not already exist. Foreign key constraints link patients to their appointments, bills, and symptom records.

4.1 Table Summary

Table	Primary Key	Key Columns
users	id (AUTO)	username, email, password (SHA-256), role
patients	id (AUTO)	name, email, phone, age, gender, address, medical_history, disease
doctors	id (AUTO)	name, specialty, phone, email
appointments	id (AUTO)	patient_id (FK), doctor, appointment_date, status, notes

billing	id (AUTO)	patient_id (FK), amount, description, bill_date, paid (0/1)
symptoms	id (AUTO)	patient_id (FK), symptoms, recommendation, analysis_date

5. System Modules:

MedCore is divided into six functional modules, each accessible from the sidebar navigation panel. Below is a detailed description of each module.

5.1 Dashboard:

The Dashboard is the landing page shown immediately after login. It provides three live summary statistics: total registered patients, the number of appointments scheduled for today, and the count of unpaid bills. A table beneath the statistics shows all of today's appointments with patient name, time, doctor, and status.

5.2 Patient Management:

This module allows staff to add new patients with complete demographic and clinical information including name, email, phone number, age, gender, home address, medical history, and the current disease or condition. Patients can be searched in real time by name, email, or phone number. Individual patient records can also be deleted from the system.

5.3 Appointment Scheduling:

Staff can schedule appointments by selecting a patient from a dropdown list and assigning a doctor — either from registered doctors in the database or by typing a doctor name manually. Appointment date and time are selected via a date-time picker. Existing appointments can be marked as 'Completed' with a single click. All appointments are displayed in a sortable table ordered by date, newest first.

5.4 Billing & Payments:

The Billing module supports creation of bills linked to individual patients, with a description field and a numeric amount. Bills are listed with their paid/unpaid status. Staff can search bills by patient name or email and mark individual bills as paid using the 'Process Payment' button. This provides a clear audit trail of all financial transactions.

5.5 Symptom Analyzer:

The Symptom Analyzer is a rule-based clinical decision support tool. The user selects one or more symptoms from six checkboxes: Fever, Cough, Headache, Nausea, Fatigue, and Shortness of Breath. The system maps each symptom to a recommended medical specialty and then queries the doctors table to identify registered doctors whose specialty matches the recommendation. Results are displayed as a formatted text recommendation.

Symptom	Recommended Specialist
Fever	General Practitioner
Cough	Pulmonologist
Headache	Neurologist
Nausea	Gastroenterologist
Fatigue	General Practitioner
Shortness of Breath	Cardiologist

5.6 Admin Settings:

The Admin Settings panel is organized into three tabs: User Management, Doctor Management, and System Settings. In User Management, an administrator can add new system accounts with a chosen role (admin, doctor, or staff). In Doctor Management, doctors can be registered with their name, specialty, phone, and email, or removed from the system. System Settings provides a one-click database backup feature that copies medical_system.db to medical_system_backup.db.

6. Security Features:

Security is a core concern in any healthcare application. MedCore implements the following measures:

- **Password Hashing:** All user passwords are hashed using SHA-256 via Python's built-in hashlib library before being stored in the database. Plain-text passwords are never saved.
- **Authentication Gate:** The application opens on a Login window. No module is accessible until valid credentials are provided.
- **Role-Based Access:** Three roles — admin, doctor, and staff — can be assigned to each user account, supporting differentiated access levels.
- **Input Validation:** Critical dialogs enforce mandatory fields (e.g., patient name + email, doctor name + specialty) and show error messages for missing or duplicate data.
- **Local Storage Only:** All data remains on the local machine. There is no internet connectivity or cloud upload, reducing the risk of remote data exposure.

7. Technology Stack:

Technology	Version / Module	Purpose
Python 3	3.8+	Core programming language for all application logic.
PyQt5	5.x	Cross-platform GUI framework; all windows and widgets.
SQLite3	Built-in	Serverless relational database for persistent local storage.
hashlib	Standard Lib	SHA-256 cryptographic hashing for secure password storage.
shutil	Standard Lib	File copy operations used for the database backup feature.
datetime	Standard Lib	Timestamp generation and formatting for appointments and billing.

8. Installation & How to Run:

MedCore has minimal dependencies and can be set up in under five minutes on any Windows, macOS, or Linux machine with Python 3 installed.

8.1 Requirements

- Python 3.8 or higher
- PyQt5 library (pip install PyQt5)
- No additional database driver or server required

8.2 Steps

Step 1 — Install PyQt5 if not already installed:

```
pip install PyQt5
```

Step 2 — Place main.py and medical_system.db in the same folder.

Step 3 — Run the application:

```
python main.py
```

Step 4 — On first launch, click Register to create the first user account, then log in.

9. Limitations & Future Improvements:

While MedCore is fully functional for small facilities, the following limitations exist in the current version:

- No network support: MedCore is a single-user application. Multi-user concurrent access requires a migration to a client-server database such as PostgreSQL or MySQL.
- No report export: There is currently no built-in PDF or Excel export for patient lists, billing summaries, or appointment history.
- No image attachments: Patient records cannot store medical images or document scans in the current version.

- Basic role enforcement: Role-based access is stored in the database but the UI does not yet hide/show menus based on the logged-in user's role at runtime.

Planned future improvements include: cloud synchronization, prescription management, patient portal, automated appointment reminders via SMS/email, and detailed financial reporting with charts.

10. Conclusion:

MedCore demonstrates how a fully functional healthcare management solution can be built using Python and PyQt5 with minimal external dependencies. The system successfully covers the core administrative and clinical workflows of a healthcare facility, including patient registration, doctor management, appointment scheduling, billing and payments, symptom analysis, and user administration. The use of SQLite3 as the storage engine ensures zero-configuration deployment — the entire system runs from a single Python file and a single database file. Security is maintained through SHA-256 password hashing, role-based user accounts, and strict input validation across all data entry forms.

MedCore provides a solid foundation for a production-grade hospital management system, and its modular design makes it straightforward to extend with additional features in the future.

— End of Report —